

Practical 3:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    printf("=== STAGE 1: Single Parent Process ===\n");
    printf("[Parent] PID: %d, PPID: %d\n", getpid(), getppid());
    printf("[Parent] Sleeping for 15 seconds. Check state using 'ps' or '/proc/%d/status'...\n\n", getpid());
    sleep(15); // State: Interruptible Sleep (S) / Waiting

    // Create child process
    pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        return 1;
    }
    else if (pid == 0) {
        // === CHILD PROCESS ===
        printf("=== STAGE 2: Child Process Active ===\n");
        printf("[Child] PID: %d, PPID: %d\n", getpid(), getppid());

        printf("[Child] Simulating CPU Work (Running - R state)...\n");
        // Active CPU loop to demonstrate Running (R) state
        for (long i = 0; i < 500000000; i++);

        printf("[Child] Entering Sleeping (S) state for 20 seconds...\n");
        sleep(20); // State: Interruptible Sleep (S) / Waiting

        printf("[Child] Terminating now (will become Zombie - Z state until parent waits)...\n");
        exit(0); // Transition to Terminated
    }
    else {
        // === PARENT PROCESS ===
        printf("=== STAGE 3: Parent Monitoring Child (PID: %d) ===\n", pid);

        printf("[Parent] Sleeping for 40 seconds to let Child finish and become a Zombie (Z)...\n");
    }
}
```

Commands:

```

bhanu@DESKTOP-DE32HQ8:~$ gcc process_states.c -o process_states
bhanu@DESKTOP-DE32HQ8:~$ ./process_states
=== STAGE 1: Single Parent Process ===
[Parent] PID: 13906, PPID: 13841
[Parent] Sleeping for 15 seconds. Check state using 'ps' or '/proc/13906/status'...

=== STAGE 3: Parent Monitoring Child (PID: 13908) ===
[Parent] Sleeping for 40 seconds to let Child finish and become a Zombie (Z)...
=== STAGE 2: Child Process Active ===
[Child] PID: 13908, PPID: 13906
[Child] Simulating CPU Work (Running - R state)...
[Child] Entering Sleeping (S) state for 20 seconds...
[Child] Terminating now (will become Zombie - Z state until parent waits)...

=== STAGE 4: Reaping Child Process ===
[Parent] Cleaned up child process. Child Zombie state resolved.
[Parent] Final sleep before exiting...

```

```

bhanu@DESKTOP-DE32HQ8:~$ nano process_states.c
bhanu@DESKTOP-DE32HQ8:~$ gcc process_states.c -o process_states
./process_states
=== STAGE 1: Single Parent Process ===
[Parent] PID: 13864, PPID: 13841
[Parent] Sleeping for 15 seconds. Check state using 'ps' or '/proc/13864/status'...

=== STAGE 3: Parent Monitoring Child (PID: 13865) ===
[Parent] Sleeping for 40 seconds to let Child finish and become a Zombie (Z)...
=== STAGE 2: Child Process Active ===
[Child] PID: 13865, PPID: 13864
[Child] Simulating CPU Work (Running - R state)...
[Child] Entering Sleeping (S) state for 20 seconds...
[Child] Terminating now (will become Zombie - Z state until parent waits)...

=== STAGE 4: Reaping Child Process ===
[Parent] Cleaned up child process. Child Zombie state resolved.
[Parent] Final sleep before exiting...

```

```

[Parent] Final sleep before exiting...
bhanu@DESKTOP-DE32HQ8:~$ top -p $(pgrep -d',' process_states)
top: option requires an argument -- 'p'
bhanu@DESKTOP-DE32HQ8:~$ ./process_states
=== STAGE 1: Single Parent Process ===
[Parent] PID: 13939, PPID: 13841
[Parent] Sleeping for 15 seconds. Check state using 'ps' or '/proc/13939/status'...

=== STAGE 3: Parent Monitoring Child (PID: 13940) ===
[Parent] Sleeping for 40 seconds to let Child finish and become a Zombie (Z)...
=== STAGE 2: Child Process Active ===
[Child] PID: 13940, PPID: 13939
[Child] Simulating CPU Work (Running - R state)...
[Child] Entering Sleeping (S) state for 20 seconds...
[Child] Terminating now (will become Zombie - Z state until parent waits)...

=== STAGE 4: Reaping Child Process ===
[Parent] Cleaned up child process. Child Zombie state resolved.
[Parent] Final sleep before exiting...

```